TEM_UW_APP

Technical description

Lehigh TEM Felzer Lab

November 2026

Contents

1 Overview **2**

2 What files are what **2**

3 Setup **2**

4 Running a case **2**

5 Configuration **3**

6 Compile-time options **3**

7 Input data **4**

8 Test **4**

9 Documentation **4**

10 Limits **5**

11 Global input data **5**

11.1 Climate forcing 5
11.2 Sources 6
11.3 Ozone 7
11.4 Nitrogen deposition 7
11.5 Land cover 7
11.6 References 8

1 Overview

Prepares and runs a case of TEM_2026. The app regrids gridded NetCDF data to the model grid, writes the NetCDF inputs and `tem_config.yaml` of the model, and runs or submits `xtem`. It also builds NetCDF inputs from public climate and land datasets.

2 What files are what

Path	Content
`case_submit.sh`	entry point, run from the case directory
`config/run_config.yaml`	all keys with defaults and comments
`src/driver.py`, `tem_climate.py`, `tem_config.py`	case builder: regridding, units, `tem_config.yaml`
`src/gen_climate_inputs.py`	downloads ERA5, gridMET, MACA or CMIP6 forcing for a box and years and writes the climate file
`src/gen_static_inputs.py`	downloads the global datasets and builds `co2.nc`, `static.nc`, `lulc.nc`, `flash.nc` and `popdens.nc` for a box
`launchers/`	local and SLURM launchers
`examples/west/`	western United States example
`docs/`	`global_data.md` (sources of the input data)
`tools/make_docs.py`	builds the two PDFs

3 Setup

```
conda env create -f env.yaml && conda activate tem_uw_app
git clone https://github.com/Lehigh-TEM-Felzer-Lab/TEM_2026
make -C TEM_2026 -j4                  # serial: build/xtem
make -C TEM_2026 mpi -j4              # MPI, for n_cpus > 1: build_mpi/xtem
export TEM_UW_APP_ROOT=$PWD
```

TEM_2026 needs g++ 9 or later, make, libyaml and libnetcdf (C). The serial build needs only `netcdf`; the MPI build needs the MPI-enabled `netcdf-mpi` with `hpcx-mpi` or `openmpi` (module names vary by site).

4 Running a case

The call hierarchy of the model is in the flowchart of TEM_2026.

1. Copy `examples/west/run_config.yaml` to an empty directory.
2. Set `tem_src` (the TEM_2026 checkout), the climate files, the box and the years. For `n_cpus` above 1 build `make mpi`; to run `xtem_competition` set `tem_exe`.
3. Run `$TEM_UW_APP_ROOT/case_submit.sh` from that directory. With `run_mode: slurm` it submits with sbatch; with `run_model: false` it only builds the case.

```
run_config.yaml   tem_config.yaml
INPUT/            climate.nc static.nc lulc.nc co2.nc (flash.nc popdens.nc)
OUTPUT/           tem.nc, written by the model
logs/
```

5 Configuration

`config/run_config.yaml` lists every key; keys you do not set take its values and unknown keys are an error.

Key	Description
`tem_src`, `tem_exe`	TEM_2026 checkout and executable (default chosen from `n_cpus` and `run_dt_units`, see Compile-time options)
`climate_file`, `netcdf_climate_in_frequency`	climate files (path, glob or list) and their time step (hourly, daily, monthly)
`static_file`, `lulc_file`, `co2_file`	static, land-cover and CO2 files in the layout of TEM_2026 `docs/io_structure.md`
`lightning_file`, `population_file`	lightning flashes and population density, needed when `veg_fire: 1`
`storm_file`, `nfert_file`	storm return intervals (`storm_disturbance: true`) and N fertilizer (`xnfertflag`), layout in TEM_2026 `docs/io_structure.md`
`varmap`	`<model variable>: <variable in your files>`; the model name is used first, the map only when the file lacks it
`horizontal_resolution`, `lon_min` to `lat_max`, `regrid_method`	model grid and regridding (`box_mean`, `bilinear`, `nearest`)
`start_year`, `run_length`, `run_length_units`	first year and duration in `days`, `months` or `years` from 1 January; the app stages climate with exactly that many steps
`spinflag`, `veg_fire`, `xnfertflag`, ...	the options of `tem_config.yaml`, one key each
`cmntecd`, `vegecd`, ...	parameter files, by default `{tem_src}/params/biogeography/`
`output_file`	native NetCDF output path, default `./OUTPUT/tem.nc`
`clmoutvars`, `temoutvars`	YAML sequences of output variable names
`run_mode`, `n_nodes`, `n_cpus`, `walltime`, `partition`, `mem`, `sbatch_options`	sbatch settings as in pyUFS; the cells are shared between the `n_cpus` MPI tasks

Climate units are read from the `units` attribute and converted. `tmin` and `tmax` can replace `tair` and `dtr`, and `dewpoint` or `huss` can replace `vpr`. Daily and hourly data are averaged to months.

Use top-level `*_dat` keys for site calibration files in `run_config.yaml`. The key is a label. TEM uses the community ID declared inside the file. Case-level calibration keys replace the default set.

```
hvc_dat: "{tem_src}/params/biogeography/hvc50_openn_dndc.yaml"
hvd_dat: "{tem_src}/params/biogeography/hvd50_openn_dndc.yaml"
```

6 Compile-time options

These are options of the TEM_2026 build, not of this app. Pass them to `make` in the TEM_2026 checkout, for example `make mpi -j16 OPEN_N=0 STORM=0`. Changing an option rebuilds the objects of that build directory.

Option	Default	Effect
`OPEN_N`	1	1 open nitrogen cycle, 0 closed
`STORM`	1	1 includes stochastic tropical storm and hurricane damage, enabled by `storm_disturbance: true` and `storm_file`; 0 omits it and rejects `storm_disturbance: true`. Prescribed land-cover disturbances are always available
`CXXFLAGS`	`-O2`	an unoptimised build gives different numbers
`CXX`	`g++`	compiler; `make mpi` uses `mpicxx`

Every build makes `xtem` (independent cohorts) and `xtem_competition` (competing cohorts). One executable runs monthly and daily steps (`run_dt_units`). Without `tem_exe` the app picks `build/xtem`, or `build_mpi/xtem` when `n_cpus` is above 1.

7 Input data

The app builds `climate.nc` from your climate files, which `src/gen_climate_inputs.py` can download; `src/gen_static_inputs.py` makes the other inputs:

```
A="--out INPUT --cache downloads --bbox -125 -110 32 49 --res 0.5"
python src/gen_climate_inputs.py --out INPUT --bbox -125 -110 32 49 --start 2000 --end 2019 gridmet
python src/gen_static_inputs.py $A co2 --start 2000 --end 2019
python src/gen_static_inputs.py $A all --year 2015 --cohorts primary secondary crops pasture urban
```

`gen_climate_inputs.py` writes `climate_<source>.nc` and `varmap_<source>.yaml` (copy it into `run_config.yaml`; set `netcdf_climate_in_frequency: daily`). `all` builds `static.nc`, `lulc.nc`, `flash.nc` and `popdens.nc` (and the files they are made from); `--cohorts` and `--min-area` select the cohorts of `lulc.nc`. Ozone needs a CEDA token (`CEDA_TOKEN`) and lightning an Earthdata Login token (`EARTHDATA_TOKEN`); all other files are downloaded by the script. Sources, download links, units and the land-cover assumptions are in docs/global_data.md. Set the `netcdf_*_in_file` keys of `run_config.yaml` to your datasets. NetCDF inputs may use any filename extension.

8 Test

`python tools/run_test.py` is one file that clones TEM_2026 and TEM_UW_APP, builds the model, downloads ERA5 (needs `~/.cdsapirc` and `cdsapi`) and the other input data, makes a global 1 degree case for one year and runs it monthly and daily. Flags such as `--competition`, `--fire`, `--all` add cases; `python tools/run_test.py -h` lists them and the docstring describes the work directory, the archive and the placeholders for ozone and N deposition.

9 Documentation

File	Content
TEM_UW_APP-Technical-Description.pdf	build options, workflow and input sources
run_config_Nov2026.pdf	`run_config.yaml` key reference

The run configuration lists keys, defaults, and meanings. `python tools/make_docs.py` builds PDF copies of the configuration reference and technical description (needs Pandoc and XeLaTeX).

10 Limits

- The app writes monthly or daily forcing (`run_dt_units` is `month` or `day`; daily needs daily forcing, `netcdf_climate_in_frequency` set to `daily`). Daily results were not compared with observations.
- Tested on the western United States example only; the inputs the app writes for it are identical to those shipped with TEM_2026.
- The global land-cover builder is not validated against the lab's earlier inputs.

11 Global input data

`src/gen_static_inputs.py` writes the input files of TEM_2026 for any bounding box and resolution (run it from the root of TEM_UW_APP). The commands below are for a 0.5 degree box over the western United States; `--bbox` takes `LON_MIN LON_MAX LAT_MIN LAT_MAX` and defaults to the globe. Needs `pip install xarray netcdf4 pandas scipy rioxarray` and `wget`.

```
A="--out INPUT --cache downloads --bbox -125 -110 32 49 --res 0.5"
python src/gen_static_inputs.py $A co2 --start 2015 --end 2024
python src/gen_static_inputs.py $A all --year 2015 --cohorts primary secondary crops pasture urban --min-area 1
```

`all` writes `elev.nc`, `soil.nc`, `wind.nc`, `ozone.nc`, `ndep.nc`, `static.nc`, `lulc.nc`, `flash.nc` and `popdens.nc`; the ozone and lightning steps are skipped with a message if their login is missing (`CEDA_TOKEN`, `EARTHDATA_TOKEN`, see below). Each step can also be run alone (`soil`, `elev`, `wind`, `ndep --start 2005 --end 2014`, `ozone`, `lightning`, `population`, `static`, `lulc`). `--cohorts` keeps only the listed cohorts (`primary`, `secondary`, `crops`, `pasture`, `urban`; the cohort dimension of `lulc.nc` follows the order given here) and `--min-area` drops cohorts smaller than that area in a cell (km2). LUH2 (6 GB) is downloaded unless `--luh2` and `--luh2-static` point to local copies.

Every step downloads its source into `--cache` once. `tools/run_test.py` runs all of them for the globe at 1 degree. `static` merges the elevation, soil, wind, ozone and N deposition files into `static.nc`. The app makes `climate.nc` from climate files such as those of `src/gen_climate_inputs.py` (see Climate forcing). The regridding is the mean of the source cells in a target cell, so the target should be coarser than the source.

11.1 Climate forcing

```
python src/gen_climate_inputs.py --out INPUT --bbox -125 -110 32 49 --start 2000 --end 2019 era5      # needs
↪  ~/.cdsapirc
python src/gen_climate_inputs.py --out INPUT --bbox -125 -110 32 49 --start 2000 --end 2019 gridmet
python src/gen_climate_inputs.py --out INPUT --bbox -125 -110 32 49 --start 2000 --end 2099 maca --model BNU-ESM
↪  --scenario rcp85
python src/gen_climate_inputs.py --out INPUT --bbox -125 -110 32 49 --start 2000 --end 2099 cmip6 --model
↪  MPI-ESM1-2-HR --scenario ssp245
```

Each source writes `climate_<source>.nc` (daily, `tmin`, `tmax`, `prec`, `nirr`, and `dewpoint` or `huss`; `tair` for ERA5 and CMIP6) with the units of the source, and `varmap_<source>.yaml`. Set `climate_file` and `netcdf_climate_in_frequency: daily` and paste the varmap into `run_config.yaml`; the app converts the units and averages to months.

Source	Data	Access
`era5`	ERA5 daily statistics, 0.25 degree, global (Hersbach et al. 2020)	Copernicus CDS dataset `derived-era5-single-levels-daily-statistics`; `--grid 1.0/1.0` requests a coarser grid
`gridmet`	gridMET, 4 km, conterminous United States, from 1979 (Abatzoglou 2013)	OPeNDAP at reacchpna.org, variables `tmmx`, `tmmn`, `pr`, `srad`, `sph`
`maca`	MACAv2-METDATA, 4 km, conterminous United States, 1950-2099 (Abatzoglou and Brown 2012)	OPeNDAP at reacchpna.org; `--model`, `--scenario` `rcp45` or `rcp85`, `--member` (default `r1i1p1`)
`cmip6`	CMIP6 daily output, model grid (Eyring et al. 2016)	Pangeo Google Cloud archive (`pip install gcsfs zarr`); `--model`, `--scenario` (for example `ssp245`) for years after 2014, `--member`

The gridMET, MACA and CMIP6 downloads were tested on a 2 x 2 degree box for 2013 and 2014 and the app converted their units. The ERA5 request needs a CDS account and was not run here. The files for nitrogen fertilizer and for yearly land-cover change are not generated.

11.2 Sources

File	Variable (unit)	Source
`co2.nc`	`co2` (ppmv)	NOAA GML Mauna Loa annual mean; SSP scenarios for later years (Meinshausen et al. 2020)
`flash.nc`	`lightning_flashes` (km-2 month-1)	LIS/OTD 0.5 degree monthly climatology HRMC v2.3.2015 (Cecil et al. 2014), flash rate per day times the days of the month; needs an Earthdata Login
`popdens.nc`	`population_density` (km-2)	WorldPop 2020 persons per 1 km pixel (WorldPop 2018), summed in each cell and divided by the cell area
`static.nc`	`elev` (m)	ETOPO 2022, 60 arc-second ice surface (NOAA NCEI 2022)
	`sand`, `silt`, `clay` (percent)	SoilGrids 2.0 at 5 km (Poggio et al. 2021), layers 0-5, 5-15 and 15-30 cm weighted by thickness, g kg-1 divided by 10
	`wind` (m s-1, by month)	WorldClim 2.1 mean wind 1970-2000 (Fick and Hijmans 2017); the height is 2 m, not 10 m
	`ozone` (AOT40, by month)	Gridded surface ozone metrics v2.7 from CEDA (Evans and Sofen 2016; Sofen et al. 2016), needs a token
	`ndep` (mg N m-2 yr-1)	ISIMIP3b NHx plus NOy deposition, mean of 2005 to 2014 (Yang et al. 2026), g N m-2 month-1 times 1000
`lulc.nc`	cohorts of each cell	LUH2 v2h (Hurtt et al. 2020); potential vegetation of Ramankutty and Foley (1999)

Sources, which the script downloads itself except LUH2 (`tools/run_test.py` fetches that with wget):

```
wget https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_annmean_mlo.txt
S=https://files.isric.org/soilgrids/latest/data_aggregated/5000m
wget $S/sand/sand_0-5cm_mean_5000.tif      # also silt, clay; 5-15 and 15-30 cm
N=https://www.ngdc.noaa.gov/thredds/fileServer/global/ETOPO2022/60s/60s_surface_elev_netcdf
wget $N/ETOPO_2022_v1_60s_N90W180_surface.nc
wget https://geodata.ucdavis.edu/climate/worldclim/2_1/base/wc2.1_10m_wind.zip
wget https://sage-public-files.s3.amazonaws.com/global-potential-vegetation/potveg_nc.tar.gz
wget https://data.worldpop.org/GIS/Population/Global_2000_2020/2020/0_Mosaicked/ppp_2020_1km_Aggregated.tif
I=https://files.isimip.org/ISIMIP3b/InputData/socioeconomic/n-deposition/histsoc
wget $I/ndep-nhx_histsoc_monthly_1850_2014.nc    # also ndep-noy
wget https://luh.umd.edu/LUH2/LUH2_v2h/states.nc                    # 5.8 GB
wget https://luh.umd.edu/LUH2/LUH2_v2h/staticData_quarterdeg.nc
```

Tested on real data: the `ndep`, CO2, soil, elevation and wind steps, the `lightning` step on the downloaded LIS/OTD file (variable `HRMC_COM_FR`, flashes km-2 day-1, dimensions found by size, annual totals of 30, 10 and 56 flashes km-2 yr-1 in Florida, Pennsylvania and the Congo) and the `population` step on the full 830 MB WorldPop raster at 1 degree for the globe (header EPSG:4326, 0.0083 degree, float32, nodata -3.4e38). The `lulc` step was tested on synthetic LUH2 files with the documented layout (descending latitude, `secma`), not on the downloaded `states.nc`; its headers were not inspected. Scenario CO2 after the observed years is not downloaded; pass a file with one `year ppmv` pair per line to `co2 --scenario` (see https://esgf-node.llnl.gov/projects/input4mips/).

11.3 Ozone

The ozone file needs a CEDA account (https://help.ceda.ac.uk/article/39-ceda-account) and an access token (https://services.ceda.ac.uk/account/token/; valid for 3 days). With the token in `CEDA_TOKEN`, `ozone` downloads the monthly 1 degree file of the dataset record https://catalogue.ceda.ac.uk/uuid/c0455ab814224a05ac74642d3d44b73e/ (archive directory http://data.ceda.ac.uk/badc/proc-earth-model/data/ozone-metrics/v2.7) and uses the variable whose name contains `aot40`. Pass `--file` and `--var` for a file you downloaded. The file name, variable and unit of AOT40 were not checked, because the download needs a login. The script averages a time axis into a 12-month climatology; check the result. The data come from monitoring stations, so cells without stations are missing and TEM skips cells with missing data.

11.4 Nitrogen deposition

`ndep` adds the NHx and NOy files of ISIMIP3b (https://doi.org/10.48364/ISIMIP.600567.1, unit g N m-2 month-1, 1850 to 2014), sums the months of `--start` to `--end` (default 2005 to 2014), averages the years and multiplies by 1000 to get mg N m-2 yr-1. The file carries the unit per N, so no further conversion is needed. For other data pass `--file`, `--var` and `--scale`. Another source is the ACCMIP multi-model deposition of Lamarque et al. (2013) (https://doi.org/10.5194/acp-13-7997-2013).

11.5 Land cover

The script writes up to seven cohorts per cell from the LUH2 fractions of one year and the cell area, in the encoding of the Hurtt cohort file of TEM_Biogeography (`cruHurtt3.2.1lulccohrts`, 2014):

Cohort	LUH2 states	`potveg`, `currentveg`, `subtype`	`agstate`	`fertflag`, `irrgflag`	`standage`
`primary`	`primf`, `primn`	potential vegetation	0	0	3000
`secondary`	`secdf`, `secdn`	potential vegetation	0	0	`secma`, 425 if absent
`crops`	`c3ann`, `c4ann`, `c3per`, `c4per`, `c3nfx`	50	1	1	0
`pasture`	`pastr`, `range`	51	2	0	0
`urban`	`urban`	48	3	1	0

For the mixed potential types 8, 14, 27 and 31 the primary and the secondary cohort are written twice, once per component, with `subtype` 9 and 10, 12 and 16, 12 and 16, and 10 and 12 and the full area in both, as in the Hurtt file. `chrtarea` is the area in km2 and `FRI` is 2000. A cohort exists where its area exceeds `--min-area` (0.001 km2). The model counts the cohorts of each cell from the cells where `standage` has data. `cohort` lists the cohort names; `potveg`, `currentveg` and `subtype` carry `flag_values` and `flag_meanings` that list the TEM vegetation types by name.

The 15 classes of Ramankutty and Foley map to TEM types (`params/biogeography/tcomm50_lulc.yaml` of TEM_2026) as 1 to 16, 2 to 18, 3 to 33, 4 to 9, 5 to 10, 6 and 7 to 4, 8 to 8, 9 to 14 (31 poleward of 23.5 degrees), 10 to 13, 11 and 12 to 15, 13 to 3, 14 to 21 and 15 to 2. The Hurtt file used TEM's own finer potential vegetation: in 3381 cells of the lower 48 states it agrees with this crosswalk in 44 percent of the cells, so results differ from the old runs where the types differ. The savanna rule, the crosswalk and the age of secondary stands are assumptions of the script; the model developers should confirm them before a production run. Tillage, disturbance and land-cover change in time are not written, so a run uses `tlulcflag: 0`.

`lulc.nc` has `cohort` as dimension and as int64 coordinate; `lat` and `lon` are float32, categorical fields int64 with `_FillValue -9999`, other fields float32. The layouts of all files are in `docs/io_structure.md` of TEM_2026.

11.6 References

- Abatzoglou, J. T. (2013) Int. J. Climatol. 33, 121-131. https://doi.org/10.1002/joc.3413
- Abatzoglou, J. T. and Brown, T. J. (2012) Int. J. Climatol. 32, 772-780. https://doi.org/10.1002/joc.2312
- Cecil, D. J., Buechler, D. E. and Blakeslee, R. J. (2014) Atmos. Res. 135-136, 404-414. https://doi.org/10.1016/j.atmosres.2012.06.028
- Evans, M. J. and Sofen, E. D. (2016) Gridded Global Surface Ozone Metrics data (1971-2015), version 2.7. CEDA. https://doi.org/10.5285/897a3958-5bfc-4311-9bb0-01134bf6aefa
- Eyring, V. et al. (2016) Geosci. Model Dev. 9, 1937-1958. https://doi.org/10.5194/gmd-9-1937-2016
- Fick, S. E. and Hijmans, R. J. (2017) Int. J. Climatol. 37, 4302-4315. https://doi.org/10.1002/joc.5086
- Hersbach, H. et al. (2020) Q. J. R. Meteorol. Soc. 146, 1999-2049. https://doi.org/10.1002/qj.3803
- Hurtt, G. C. et al. (2020) Geosci. Model Dev. 13, 5425-5464. https://doi.org/10.5194/gmd-13-5425-2020
- Lamarque, J.-F. et al. (2013) Atmos. Chem. Phys. 13, 7997-8018. https://doi.org/10.5194/acp-13-7997-2013
- Meinshausen, M. et al. (2020) Geosci. Model Dev. 13, 3571-3605. https://doi.org/10.5194/gmd-13-3571-2020
- NOAA NCEI (2022) ETOPO 2022 15 Arc-Second Global Relief Model. https://doi.org/10.25921/fd45-gt74
- Poggio, L. et al. (2021) SOIL 7, 217-240. https://doi.org/10.5194/soil-7-217-2021
- Ramankutty, N. and Foley, J. A. (1999) Global Biogeochem. Cycles 13, 997-1027. https://doi.org/10.1029/1999GB900046
- Sofen, E. D. et al. (2016) Earth Syst. Sci. Data 8, 41-59. https://doi.org/10.5194/essd-8-41-2016
- WorldPop (www.worldpop.org) and CIESIN, Columbia University (2018) Global High Resolution Population Denominators Project. University of Southampton. https://doi.org/10.5258/SOTON/WP00647
- Yang, J., Tian, H. and Büchner, M. (2026) ISIMIP3b Simulated Historical Nitrogen Deposition Input Data, v1.1. ISIMIP Repository. https://doi.org/10.48364/ISIMIP.600567.1